

Screenshot Descriptions & Steps

Screenshot 1: 01_assembler_no_errors.png

Description: Assembler status showing successful compilation with 0 warnings and 0 errors.

Screenshot 2: 02_initial_load.png

Description: Program loaded in Easy68K simulator, showing initial state before execution with Automatic Disabled.

Screenshot 3: 03_prog1_running_count2.png

Description: Stopwatch test program (prog1) running with counter displaying "2" on 7-segment display.

Screenshot 4: 04_prog1_select_code.png

Description: Code showing bra prog1 active for selecting stopwatch test program. Automatic Enabled, timer running.

Screenshot 5: 05_prog2_select_code.png

Description: Code showing bra prog2 active for selecting radiation monitor test program.

Screenshot 6: 06_prog2_result.png

Description: Radiation monitor (prog2) result after 8 seconds. Display shows c/8 value. RH LED ON (c>1000). LH LED OFF confirms mutex worked correctly with no race conditions.

Screenshot 7: 07_memory_vector_table.png

Description: Memory view showing interrupt vector table. Address \$80 contains TRAP #0 vector pointing to syscall_handler.

Screenshot 8: 08_enable_exceptions.png

Description: Options menu showing "Enable Exceptions" is checked - required for TRAP instructions to work.

Screenshot 9: 09_timer_irq_settings.png

Description: Hardware timer configuration with IRQ level 1 enabled, Auto Interval 1000ms, Automatic Enabled.

Screenshot 10: 10_code_flih.png Description: First Level Interrupt Handler (FLIH) code. Saves all registers (D0-D7, A0-A6), stack pointer, status register, and program counter to the current task's TCB.

Screenshot 11: 11_code_disp.png Description: Dispatcher code. Restores all registers from TCB and uses RTE to resume the task. This is the reverse of FLIH.

Screenshot 12: 12_code_scheduler.png Description: Round-robin scheduler. Advances rdytcb to the next task in the circular ready list.

Screenshot 13: 13_code_mutex.png Description: sys_wait_mutex implementation. If mutex is locked, blocks the task and performs context switch. If free, locks it and continues.

Screenshot 14: 14_code_timer_isr.png Description: Timer ISR - the main driver of time-slicing. Calls FLIH, update_waiters, scheduler, then dispatcher.

Screenshot 15: 15_code_create_validation.png

Description: sys_create validation code showing address range checking (\$2000-\$8000) for both task address and stack pointer. Invalid addresses result in -1 return code.

Screenshot 16: 16_code_delete_cleanup.png

Description: sys_delete mutex cleanup code. If task holds mutex (value=0) when deleting, mutex is automatically released (set to 1) to prevent system-wide deadlock.

Screenshot 17: 17_code_wait_time_check.png

Description: sys_wait_time zero check code. If wait time is ≤ 0 , function returns immediately without blocking, preventing unnecessary context switch.

Screenshot 18: 18_code_signal_doublecheck.png

Description: sys_signal_mutex double-signal prevention. If mutex is already free (value=1), subsequent signals are silently ignored to maintain invariant.

Screenshot 19: 19_prog3_result.png

Description: Error handling test (prog3) result. Display shows "E" pattern (\$79), confirming invalid address detection returned -1 as expected. LEDs remain OFF indicating test passed.

Screenshot 20: 20_prog3_select_code.png

Description: Code showing bra prog3 active for selecting error handling test program.

Steps Followed (For Documentation)

Setup Steps:

1. Open RTOS_Skeleton.X68 in Easy68K Editor
2. Assemble code (F9) - verify 0 errors
3. Click Execute (F10) to open simulator
4. Go to Options → Enable Exceptions (check it)
5. Go to View → Hardware to open hardware window
6. Set IRQ Level to 1
7. Set Auto Interval to 1000 mS
8. Enable the checkbox for IRQ 1

Testing prog1 (Stopwatch):

1. Ensure bra prog1 is active in code
2. Assemble and Execute
3. Press F9 (Run)
4. Observe counter incrementing on 7-segment display
5. Toggle Switch 0 to start/stop (behavior varies in simulator)

Testing prog2 (Radiation Monitor):

1. Comment out bra prog1, uncomment bra prog2
2. Reassemble and Execute
3. Press F9 (Run)
4. Wait 8 seconds
5. Observe: Display shows c/8 value, RH LED ON, LH LED OFF (no mutex error)

Testing prog3 (Error Handling Verification):

1. Comment out bra prog2, uncomment bra prog3
2. Reassemble and Execute
3. Press F9 (Run)

4. Observe: Display shows "E", LEDs remain OFF
5. Result: Confirms sys_create correctly rejected invalid address (\$1000) and returned -1

Code Sections to Screenshot

Key RTOS Functions (Code Screenshots):

1. flih: First Level Interrupt Handler - saves context - 10_code_flih.png
2. disp: Dispatcher - restores context - 11_code_disp.png
3. scheduler: Round-robin scheduler - 12_code_scheduler.png
4. sys_wait_mutex: Mutex wait implementation - 13_code_mutex.png
5. timer_isr: Timer ISR - drives time-slicing - 14_code_timer_isr.png

Error Handling Functions (Code Screenshots):

6. sys_create validation: Address range checking - 15_code_create_validation.png
7. sys_delete cleanup: Mutex release on task exit - 16_code_delete_cleanup.png
8. sys_wait_time check: Zero/negative time validation - 17_code_wait_time_check.png
9. sys_signal_mutex guard: Double-signal prevention - 18_code_signal_doublecheck.png

Testing Procedure

Easy68K Simulator Setup:

1. Open RTOS_Skeleton.X68 in Easy68K Editor/Assembler
2. Assemble (F9) - confirm 0 errors, 0 warnings
3. Execute (F10) to launch simulator
4. Enable Options → Enable Exceptions
5. Open View → Hardware window
6. Configure timer: IRQ Level 1, Auto Interval 1000ms, Automatic Enabled

Test Program 1 - Stopwatch: The stopwatch demonstrates timer-based task switching. Task 0 displays and increments a counter on the 7-segment display. Task 1 monitors Switch 0 to toggle the running state. The counter increments once per second, proving the timer ISR and sys_wait_time system call function correctly.

Test Program 2 - Radiation Monitor: The radiation monitor tests mutex functionality with three concurrent tasks. Tasks 0 and 1 increment separate counters (a and b) and a shared counter (c) protected by mutex. Task 2 waits 8 seconds, then displays $c/8$ and checks mutex correctness. The formula $a + b - c$ should equal 0 or 1 if the mutex works correctly. If it exceeds 1, the LH LED lights indicating a race condition. In our tests, the LH LED remained OFF, confirming successful mutex protection.

Test Program 3 - Error Handling Verification: This test validates the RTOS's defensive programming features. It attempts to create a task at address \$1000 (below the valid user space boundary of \$2000). The `sys_create` function should detect this invalid address and return -1. If the test succeeds, the display shows "E" and LEDs remain OFF. If it fails (address accepted or wrong error code), both LEDs light up. This confirms the RTOS correctly validates parameters and prevents execution of non-existent code.

Results: - prog1: Counter increments correctly, demonstrating working timer interrupts and context switching - prog2: LH LED OFF after 8 seconds, confirming mutex correctly protects shared data - prog3: Display shows "E" with LEDs OFF, confirming error handling detects invalid addresses

ERROR HANDLING AND ROBUSTNESS

Parameter Validation

The RTOS validates all user inputs to prevent system crashes:

****sys_create - Address Validation:****

- Task address must be \$2000-\$7FFF (user code space)
- Stack pointer must be \$2000-\$8000 (valid stack area)
- Invalid addresses return -1 error code
- Prevents executing non-existent code or corrupting system memory

****sys_wait_time - Zero Check:****

- Wait time ≤ 0 returns immediately without blocking
- Prevents unnecessary context switch overhead
- Avoids undefined behavior from negative time

Unusual Eventualities Handled

****Task Deleted While Holding Mutex:****

- Problem: Mutex stays locked forever (deadlock)
- Solution: sys_delete checks mutex state and releases it before task removal
- Impact: Prevents system-wide deadlock, other tasks can acquire mutex

****Double Mutex Signal:****

- Problem: Signaling free mutex could corrupt value (set to 2+)
- Solution: sys_signal_mutex checks if already free and ignores redundant signals
- Impact: Maintains mutex invariant (value always 0 or 1)

****Empty Ready List:****

- Problem: Scheduler called when no tasks ready
- Solution: Scheduler checks for NULL and returns without modification
- Impact: Current task continues until others become ready

****All TCBs In Use:****

- Problem: User tries to create 9th task with only 8 TCBs
- Solution: sys_create searches all TCBs, returns -1 if none free
- Impact: Prevents memory corruption from array overflow

Evidence of Independent Study

****Error Return Codes:****

- Source: POSIX system calls (read, write, fork)
- Application: sys_create returns 0 on success, -1 on error
- Allows caller to detect and handle failures gracefully

****Resource Cleanup on Exit:****

- Source: Linux kernel `do_exit()` function
- Application: Automatic mutex release in `sys_delete`
- Prevents "zombie locks" that would deadlock the system

****Defensive Programming:****

- Source: "The Practice of Programming" by Kernighan & Pike
- Principles: Never trust user input, fail gracefully, maintain invariants
- Applied throughout RTOS (all parameters validated before use)

****Mutex Ownership Transfer:****

- Source: FreeRTOS priority inheritance documentation
- Application: `sys_signal_mutex` passes mutex directly to waiting task
- Avoids race where third task could steal mutex before waiter runs

Testing Results

Test	Input	Expected	Result	Screenshot
-----	-----	-----	-----	-----
prog1	Normal operation	Counter increments	 PASS	3
prog2	Mutex protection	LH LED OFF	 PASS	6
prog3	Invalid address \$1000	Return -1, show "E"	 PASS	19